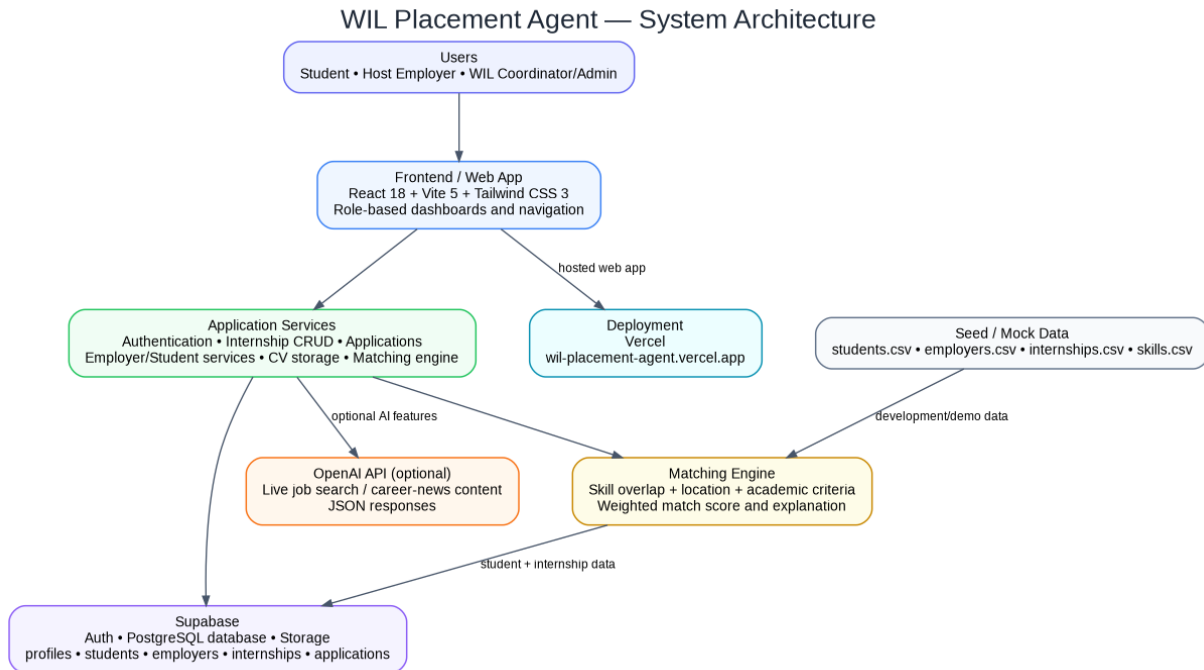


WIL PLACEMENT AGENT

System Documentation

Work-Integrated Learning (WIL) Placement Management Portal



Deployed application: <https://wil-placement-agent.vercel.app/>

Prepared from the supplied source code, database schema, architecture notes, and use-case requirements.

1. System Overview

The WIL Placement Agent is a web-based Work-Integrated Learning placement portal designed to connect students with workplace learning opportunities and to support employers and WIL coordinators in managing the placement process. The system provides separate role-based experiences for students, host employers, and administrators/WIL coordinators.

The supplied implementation uses React for the user interface, Vite for development/build tooling, Tailwind CSS for styling, and Supabase for authentication, PostgreSQL data management and CV storage. A matching engine evaluates student skills against internship requirements and produces a match percentage. Optional OpenAI integration is present for live job-search and career-news functionality.

2. Problem Statement

University Work-Integrated Learning placement processes can become difficult to manage when student profiles, CVs, internship opportunities, applications, employer information and placement decisions are handled across disconnected spreadsheets, emails and manual processes. This can make it harder for students to identify suitable opportunities, for employers to review applicants consistently, and for WIL coordinators to monitor placement activity.

The WIL Placement Agent addresses this problem by providing a central online platform where students can maintain placement-related information and apply for opportunities, employers can publish and manage internships and review applicants, and coordinators/admin users can oversee employers and matching activities.

3. Objectives

- Provide a single portal for WIL placement activities.
- Separate functionality according to Student, Host Employer, and WIL Coordinator/Admin roles.
- Allow employers to publish internship opportunities and manage applicants.
- Allow students to browse opportunities, compare their skills with requirements, and apply with a CV.
- Provide a structured matching engine based on skills and other placement criteria.
- Store user, internship and application information in a central database.
- Provide CV upload and secure signed-link access through Supabase Storage.
- Support optional AI-assisted content features through an external OpenAI API integration.

4. Target Users and Roles

Role	Main Responsibilities	Key Portal Functions
Student	Find and apply for WIL opportunities.	Register/login, browse internships, view skill matches, apply with CV, track applications.
Host Employer	Provide workplace learning	Post internships, manage

	opportunities and assess applicants.	listings, review applicants, inspect skills/CVs, schedule interviews, respond to applications.
WIL Coordinator / Admin	Oversee the placement programme and matching process.	Manage employers, view platform information, execute matching, support compliance/reporting functions.

5. Core System Features

Feature	Description
Authentication and role routing	Supabase Auth is used to restore sessions and route authenticated users to the correct role-specific dashboard.
Student placement discovery	Students can search and filter internship listings and see skill-match percentages for required skills.
Online applications	Students can apply to internships and upload a PDF CV. Applications are stored against the student and internship.
Employer internship management	Employers can create and manage internship listings and view applicants for their listings.
Applicant requirements scan	The employer interface calculates the overlap between applicant skills and required internship skills.
Interview/application response	Employers can respond to applications, schedule an interview date, or decline an application.
Automated matching	The matching engine considers skills, location and academic-average requirements and calculates a weighted score.
AI-assisted content	OpenAI integration can provide live job-search and career-news content when an API key is configured.
CV storage	CV PDFs are stored in a Supabase Storage bucket and exposed through signed URLs.

6. Technologies and Frameworks Used

Technology	Use in the System
React 18.3.1	Frontend framework/library used to build component-based interfaces and role-specific pages.
Vite 5.3.1	Frontend development server and production build tool.
Tailwind CSS 3.4.4	Utility-first CSS framework used for layout, styling, spacing, responsive UI and visual states.
Supabase JS 2.45.0	Client library used for authentication, PostgreSQL data access and Storage.
Supabase Auth	Handles login/session restoration and authenticated user identity.
Supabase PostgreSQL	Stores profiles, students, employers, internships and applications.
Supabase Storage	Stores student CV PDF files in the cvs bucket.
OpenAI API	Optional external AI integration for live job-search and career-news content; the code uses JSON-formatted responses.
JavaScript / JSX	Primary implementation language for application

	logic and React components.
Vercel	Deployment platform for the hosted web application.
Graphviz / PlantUML-style notation	The supplied use-case requirement uses UML-style actors/use cases; this document renders the diagram as a clean UML-inspired figure.

7. System Architecture

The system follows a modular client-side architecture. The React application contains role-based pages, shared UI components, service modules for Supabase operations, a matching utility, and optional AI service calls. Supabase acts as the main backend platform for authentication, relational data and file storage.

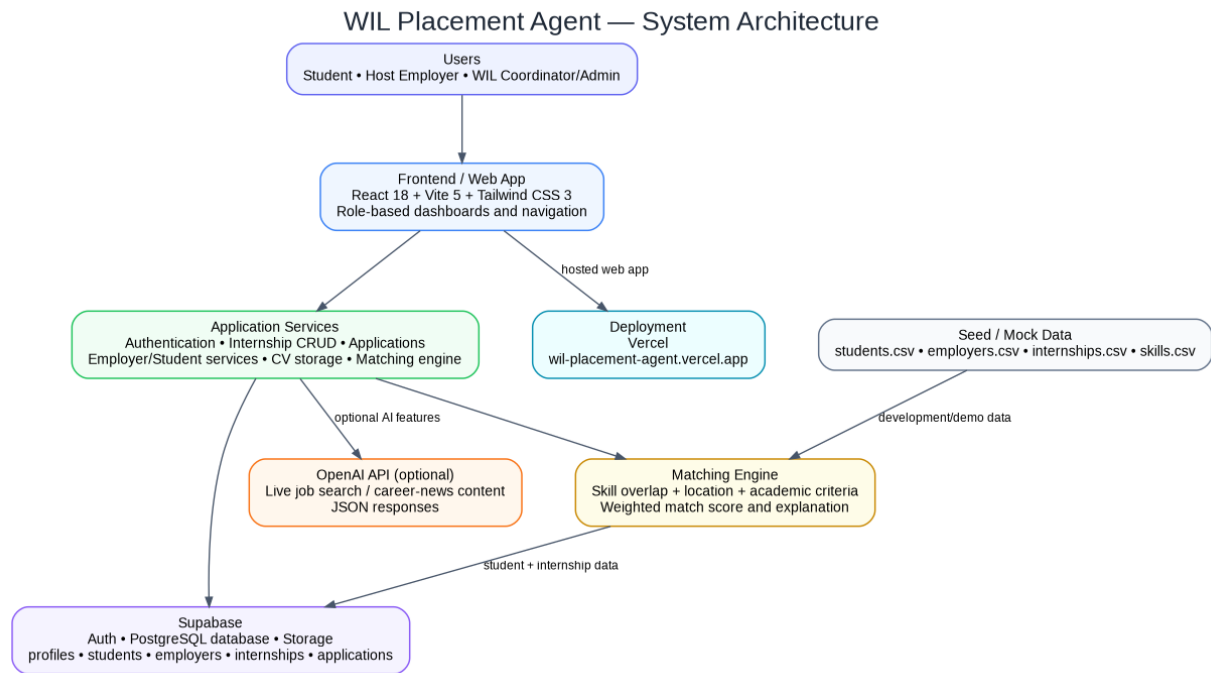


Figure 1: High-level architecture derived from the supplied project structure. The repository also contains a backend directory prepared for a future Node/Express API layer; the current application primarily communicates with Supabase directly from the React client.

7.1 Frontend Layer

- App.jsx restores the Supabase session and routes users according to their role.
- Role-specific pages are grouped under pages/admin, pages/student and pages/employer.
- Shared UI elements are provided through components/ui and the role-aware Sidebar.
- Tailwind CSS classes provide the responsive visual presentation.

7.2 Service/Data Layer

- services/auth.js manages authentication-related operations.
- services/internships.js, applications.js, employers.js and students.js encapsulate database operations.
- services/storage.js handles CV uploads to the Supabase cvs bucket.

- lib/supabase.js provides the shared Supabase client.

7.3 Matching and AI Layer

- matchingEngine.js provides skillMatch(), runMatching() and live student-to-internship matching logic.
- The live matching function checks skill overlap, location compatibility and academic-average requirements.
- The implemented weighted score uses 50% skills, 30% location and 20% academic-average criteria.
- The optional OpenAI service is used for live job-search and career-news functionality; it is not the same thing as the deterministic placement matching algorithm.

8. Database and Data Model

The supplied Supabase schema defines five principal relational tables:

Table	Purpose
profiles	Stores the authenticated user's profile, name, email and role.
students	Stores student-specific information such as programme/major, year, average mark, skills and location.
employers	Stores employer/company information and location.
internships	Stores workplace opportunities posted by employers, including type, duration, location and required skills.
applications	Links students to internships and stores application status, dates, CV URL, motivation and notes.

The schema enables one profile to represent a student or employer through role-specific records, while internships belong to employers and applications link students to internships. Row Level Security (RLS) is enabled on all five principal tables in the supplied schema.

9. Use Case Diagram

The following diagram represents the actors and use cases supplied for the WIL Placement Agent. The Host Employer and WIL Coordinator labels are retained from the requested specification.

WIL Placement Agent Portal — Use Case Diagram

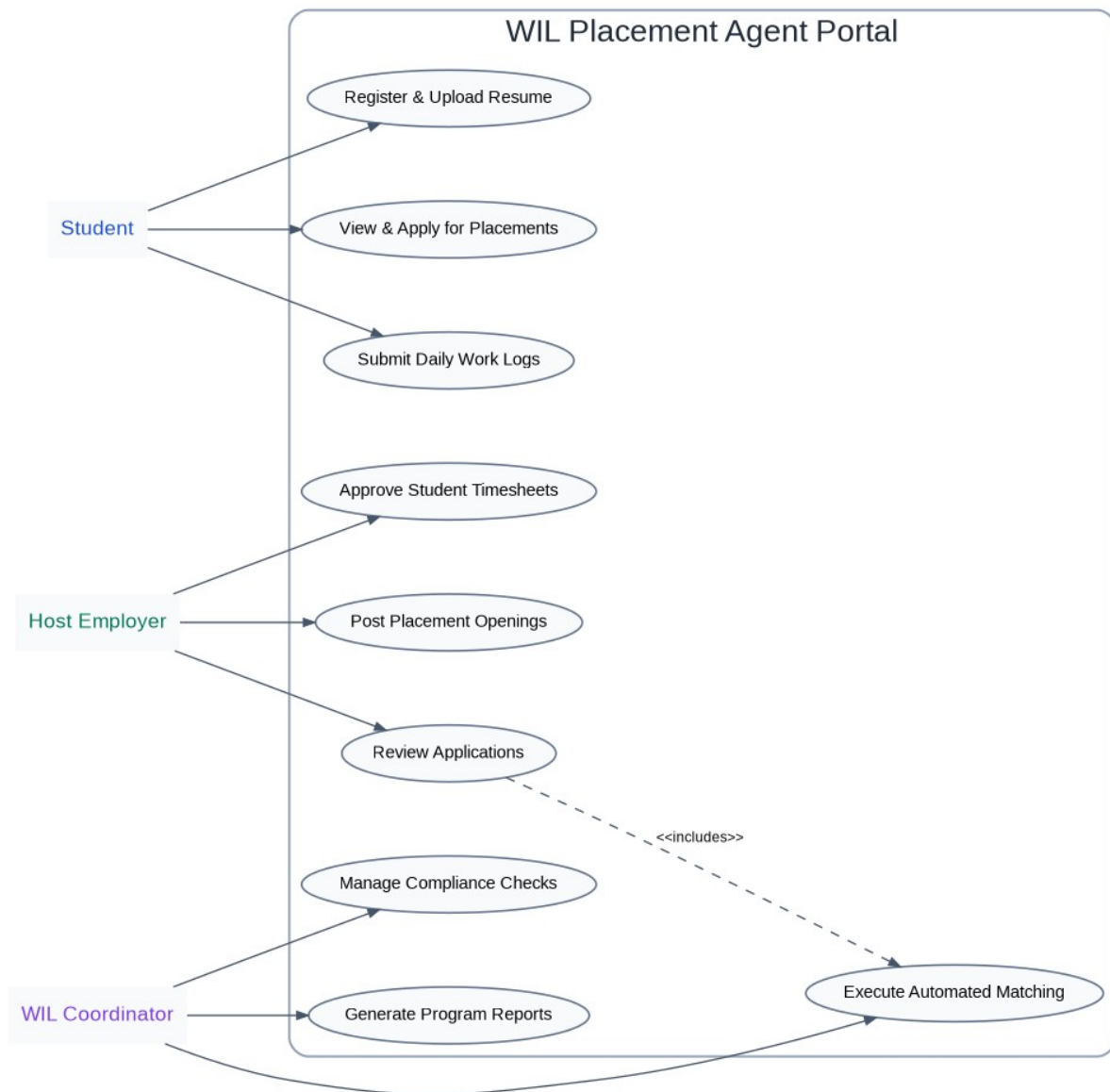


Figure 2: WIL Placement Agent use-case diagram.

Note: the requested specification shows Review Applications including Execute Automated Matching. In the implementation, matching is also exposed as a coordinator/admin function through the WIL Matching page.

10. Main User Workflows

10.1 Student Workflow

- Student registers/logs in.
- The application restores the Supabase session and loads the student's profile.
- The student opens Available Internships and searches/filters listings.
- The system calculates skill overlap against each internship's required skills.

- The student selects an internship and submits an application with a PDF CV.
- The CV is uploaded to Supabase Storage and a signed URL is stored with the application.
- The student can view application status and interview information.

10.2 Host Employer Workflow

- Employer logs in through the employer authentication flow.
- Employer posts an internship with role, location, duration and required skills.
- Applicants are retrieved for the employer's own listings.
- The employer can review programme/year/average information, skills and CV access.
- The system displays a requirements scan based on skills overlap.
- The employer can schedule an interview or decline an application.

10.3 Coordinator/Admin Workflow

- Coordinator/admin logs in.
- The dashboard provides platform-level access.
- Employers can be managed/registered.
- The WIL Matching page can run matching logic using student and internship data.
- The supplied use-case specification also identifies compliance management and programme reporting as coordinator responsibilities.

11. Placement Matching Logic

The matching engine is implemented in `src/utils/matchingEngine.js`. It provides both a simple skill-overlap function and a richer student-to-internship matcher.

- Skill percentage = $\text{matched required skills} \div \text{total required skills} \times 100$.
- A live student is considered to satisfy the skills criterion when at least 50% of required skills are matched, unless no skills are required.
- Location is considered a match when the stored/preferred student location is compatible with the internship location or the student location is empty.
- If an internship has a minimum academic average, the student's average must meet or exceed it.
- The weighted score is calculated as skills 50% + location 30% + academic average 20%.
- The matcher returns matched skills, missing skills, qualification status, criteria results and an explanatory summary.

This approach gives the portal a transparent rule-based matching foundation. It can later be replaced or augmented with a server-side recommendation service without changing the overall user workflow.

12. AI Integration

The project includes an OpenAI integration in `src/services/ai.js`. The current implementation calls the OpenAI Chat Completions endpoint when `VITE_OPENAI_API_KEY` is configured and requests JSON-formatted results. The architecture notes indicate that this is used for live South African job-search and career-news content.

The code also contains a planned Claude/Anthropic integration stub, but that integration is not implemented in the current source. The deterministic WIL matching engine should therefore be distinguished from the optional generative AI features.

Security consideration: the supplied architecture documentation explicitly notes that the OpenAI API key is currently accessed client-side. For production use, AI API calls should be moved to a protected backend service or serverless/edge function so that secret API credentials are not exposed to browsers.

13. Security and Compliance Considerations

- Supabase Auth is used for authenticated user sessions.
- Supabase Row Level Security is enabled on profiles, students, employers, internships and applications.
- CV files are stored in Supabase Storage rather than embedded in database records.
- Application records use foreign keys and unique(student_id, internship_id) to prevent duplicate applications to the same internship.
- The application validates that uploaded CVs are PDFs and limits the file size to 5 MB in the student application interface.
- Production deployment should keep API secrets server-side and use short-lived signed URLs where appropriate for sensitive documents.
- Access policies should be tested so that students can access only their own sensitive records and employers can access applicant data only for their own listings.

14. Deployment

The supplied project is deployed as a Vercel-hosted web application. The deployment referenced for this documentation is:

<https://wil-placement-agent.vercel.app/>

The package scripts support local development with Vite (npm run dev), production builds with Vite (npm run build), and local preview of the production build (npm run preview).

15. Current Limitations and Future Enhancements

- The repository contains a prepared backend directory, but the current architecture primarily performs data/API operations from the React client.
- The OpenAI API integration is client-side and should be moved behind a secure server-side layer for production.
- Some coordinator use cases in the requested UML specification—particularly compliance checks, daily work logs/timesheet approval and programme reporting—are represented in the documentation/use-case model but are not fully represented by the current database schema and page set.
- The matching engine currently uses explicit rules and weighted scoring; an advanced recommendation model could be added later.
- Automated notifications (email/SMS/WhatsApp), richer audit logging, placement completion workflows and analytics could extend the system.

- Role-specific RLS policies should be reviewed and tested against the exact production authorization requirements.

16. Conclusion

The WIL Placement Agent provides a structured digital foundation for managing Work-Integrated Learning placements. Its role-based React interface, Supabase-backed data model, CV storage, application management and rule-based matching engine address key parts of the placement lifecycle. The modular service structure also creates a clear path for future backend hardening, advanced AI recommendations, compliance workflows and reporting.

Appendix A — Project Structure

The supplied repository is organized around a modular frontend and supporting backend/database folders. Key areas include:

- `src/pages/auth` — authentication and landing screens
- `src/pages/student` — student dashboard, internships and applications
- `src/pages/employer` — employer dashboard, listings and applicant review
- `src/pages/admin` — administration and WIL matching
- `src/services` — Supabase CRUD, authentication, storage and AI services
- `src/utils/matchingEngine.js` — matching logic and development data
- `database/schema/tables.sql` — Supabase relational schema
- `data/*.csv` — mock/development datasets
- `backend/` — prepared structure for a future Node/Express API layer